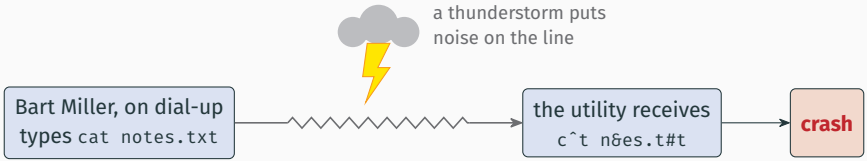
Applications of Reinforcement Learning to Emulator-Based Stateful Greybox Fuzzing

Bastian Engel

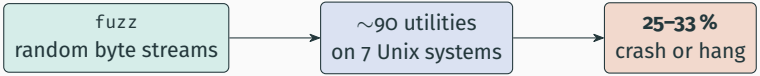
August 31, 2026

Karlsruhe Institute of Technology · Fraunhofer IOSB
Reviewer: Prof. Dr.-Ing. Jürgen Beyerer · Advisor: M.Sc. Mark Leon Giraud

The origins of fuzzing (1988)

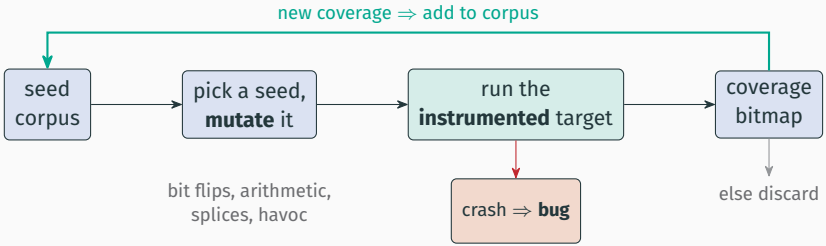


the storm mutates valid input at random — Miller turns the accident into the program fuzz



random corruption of (valid) input crashes a third of the programs of a mature OS [Miller et al. 1990]

Today: coverage-guided fuzzing

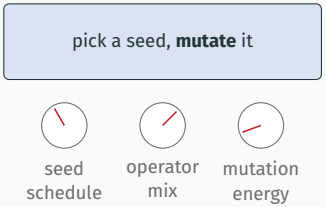


AFL (2013) made coverage feedback the standard — maintained today as **AFL++** (2020);
OSS-Fuzz (2016–) runs it at scale: >13 000 security vulnerabilities, >50 000 bugs, 1000 projects

no model of the target anywhere in the loop — it wins by raw throughput: 10^3–10^4 executions per second, natively

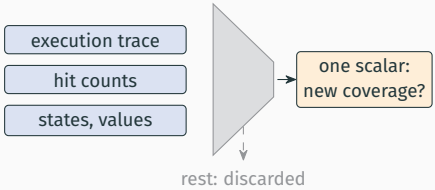
Motivation: two problems with the loop

1 — hand-tuned heuristics



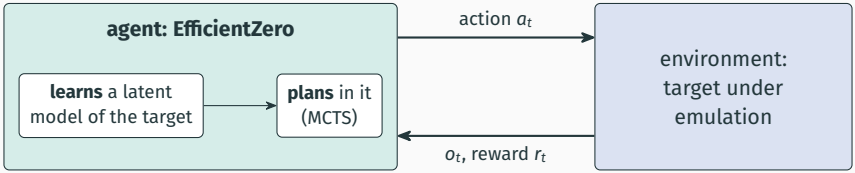
the optimal configuration is target-dependent [Woo et al. 2013; Böhme et al. 2016]

2 — information discarded



every run produces a rich behavioral signal which gets reduced to a single novelty scalar

Idea: a fuzzer that acts as a reinforcement learning agent



fuzzing is sequential decision making, which RL targets

EfficientZero (AlphaZero lineage): superhuman Atari/Chess/Go performance with high sample efficiency

To the best of our knowledge

the first fuzzer that plans with a *learned* model of the target's behavior.

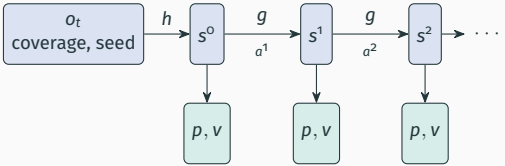
Research questions

RQ1 **Learnability** — can EfficientZero learn a useful policy from coverage-shaped rewards?

RQ2 **Action space** — program *inputs* as actions, or *mutations on a corpus*?

RQ3 **Planning** — does planning with the learned model actually help?

EfficientZero (1): learning a model of the target

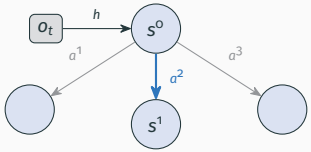


- h — representation (**observation encoding**)
- g — dynamics (**model behavior**)
- v — value function (**how good is a state?**)
- p — action prior (**search guiding**)

latents are learned, not hand-designed — and no simulator access anywhere: the model comes from trajectories alone

EfficientZero (2): planning using the learned model

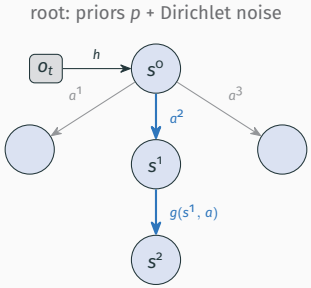
root: priors p + Dirichlet noise



1. **select** — descend the tree by pUCT

$$\mathrm{pUCT}(s,a) = \underbrace{Q(s,a)}_{\substack{\text{value}\\\text{estimate}}} + c \cdot \underbrace{P(s,a)}_{\substack{\text{prior}\\\text{from } p}} \cdot \underbrace{\frac{\sqrt{N(s)}}{1+N(s,a)}}_{\substack{\text{fades with}\\\text{visits } N(s,a)}}$$

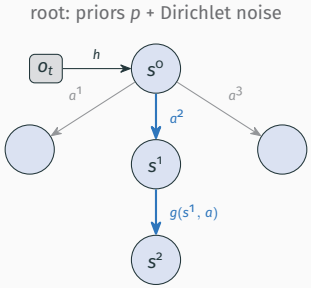
EfficientZero (2): planning using the learned model



$$\mathrm{pUCT}(s,a) = \underbrace{Q(s,a)}_{\text{value estimate}} + c \cdot \underbrace{P(s,a)}_{\text{prior from } p} \cdot \underbrace{\frac{\sqrt{N(s)}}{1+N(s,a)}}_{\text{fades with visits } N(s,a)}$$

1. **select** — descend the tree by pUCT
2. **expand** — apply $g(s,a)$

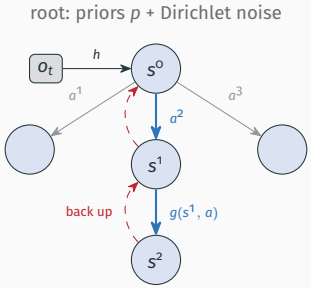
EfficientZero (2): planning using the learned model



$$\mathrm{pUCT}(s,a) = \underbrace{Q(s,a)}_{\text{value estimate}} + c \cdot \underbrace{P(s,a)}_{\text{prior from } p} \cdot \underbrace{\frac{\sqrt{N(s)}}{1+N(s,a)}}_{\text{fades with visits } N(s,a)}$$

1. **select** — descend the tree by pUCT
2. **expand** — apply $g(s,a)$
3. **evaluate** — score the new node using v

EfficientZero (2): planning using the learned model

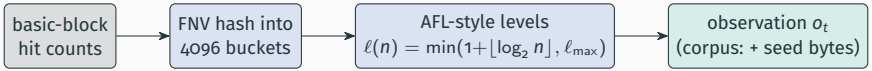


$$\mathrm{pUCT}(s,a) = \underbrace{Q(s,a)}_{\text{value estimate}} + c \cdot \underbrace{P(s,a)}_{\text{prior from } p} \cdot \underbrace{\frac{\sqrt{N(s)}}{1+N(s,a)}}_{\text{fades with visits } N(s,a)}$$

1. **select** — descend the tree by pUCT
2. **expand** — apply $g(s,a)$
3. **evaluate** — score the new node using v
4. **back up** — update value estimate

repeat for N=200 simulations (baseline)

Fuzzing as an MDP: observations and reward



Reward: breadth + depth

$$r_t = \lambda \Big[\underbrace{\textstyle\sum_i \big(\ell(c_{t,i}) - \ell(u_{t,i})\big)_+}_{\text{breadth: novelty vs. union } u_t} + \omega \underbrace{\big(\sigma(c_t) - d_t\big)_+}_{\text{depth: beat record } d_t} \Big], \qquad \sigma(c) = \textstyle\sum_i \ell(c_i)$$

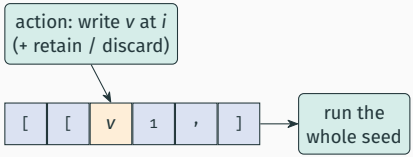
reward only *new* coverage to force exploration

Two action-space formulations

input-level: actions *are* the input

one byte or protocol message per step
the policy is a generative model of the input language

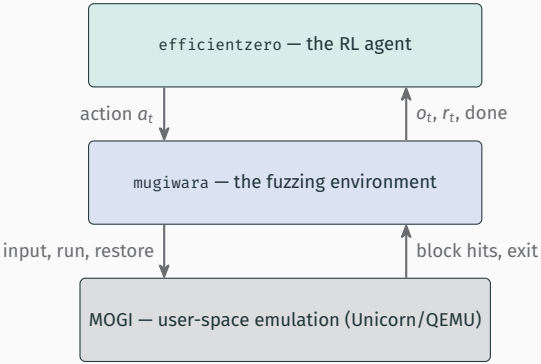
corpus-level: actions mutate a seed



deterministic byte writes on a single fixed-length seed

EfficientZero needs a **discrete, deterministic** action space of tractable size

System architecture



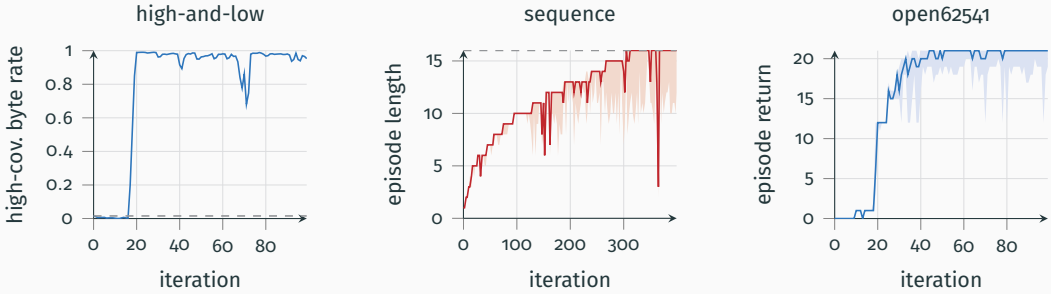
- implemented in Rust using `burn`
- `efficientzero` is generic, not bound to fuzzing

Evaluation targets

Setting	Target	Role	Actions	$\lvert\mathcal{A}\rvert$
input	high-and-low	bandit sanity check	input bytes	256
input	sequence	16-byte passcode, POMDP	bytes `[A-Z]`	26
input	open62541	real OPC UA server, reach session	protocol messages	15
corpus	cJSON	**ablation: 15 arms $\times$ 4 seeds**	position + byte write	256
corpus	libxml2	transfer of cJSON config, 400 iters	position + byte write	256
corpus	picohttpparser	transfer of cJSON config, 400 iters	position + byte write	256

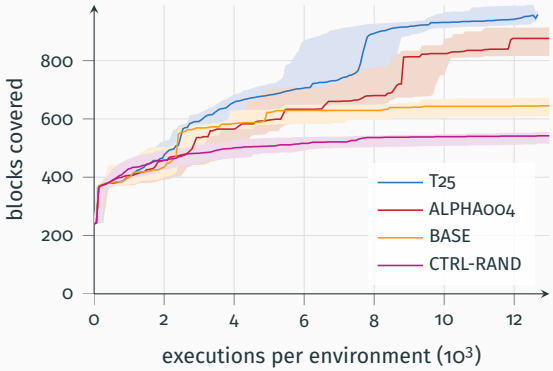
- 4 PRG seeds $\times$ 128 environments on one L40S — cJSON 1.66M executions ($\sim$21 h), transfers 3.33M ($\sim$42 h)
- `CTRL-RAND`: random search at an identical execution budget

RQ1: the learner solves all three input-level targets



- **high-and-low**: paying-byte rate $>$ 0.99 (random: 0.016)
- **sequence**: recovers the full 16-byte passcode
- **open62541**: `hel` $\rightarrow$ `opn` $\rightarrow$ `create` $\rightarrow$ `activate` — return 21 of 22

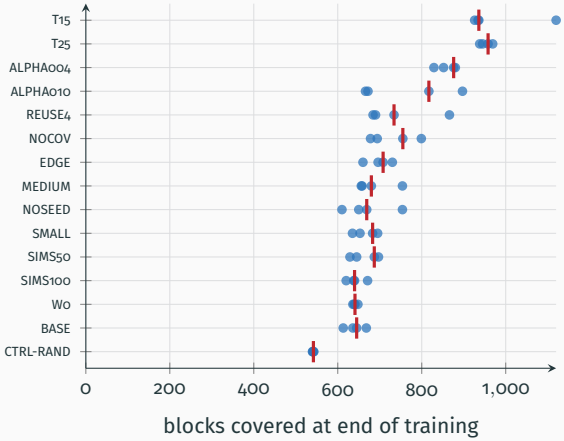
RQ1: planning beats random search



cJSON, identical execution budget, medians over 4 seeds $\times$ 128 environments:

- bootstrap corpus: 240 blocks
- random control: 542
- baseline config: 645
- best arm (T25): 958 — 1.77$\times$ the random control

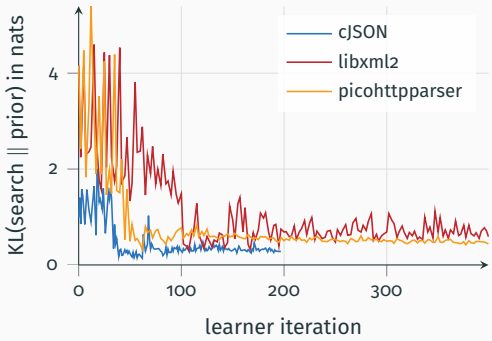
Ablation: exploration is most important



15 arms × 4 seeds; dots are seeds, bars medians.

- **constant high temperature wins**: `T25` 958, `T15` 936 vs. baseline 645
- scaled Dirichlet noise adds more: `ALPHA004` 876
- most other knobs move less than the within-arm seed spread

RQ3: planning helps early, then fades



How far MCTS moves the policy away from the network prior:

- falls near-monotonically on all targets (cJSON 0.99 $\rightarrow$ 0.28 nats)
- planning contributes most in the first iterations, little at the end

Simulation-count arms (cJSON):

	SIMS50	SIMS100	BASE
blocks	687	640	645
hours	7.6	11.5	21.3

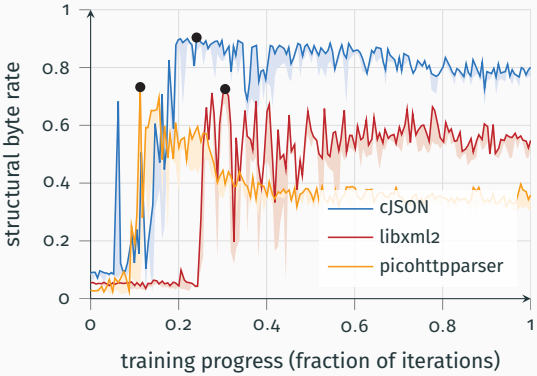
Coverage is flat across 4$\times$ less search while wall-clock scales linearly with it — on targets this small, cheap planning is enough.

The agent discovers input structure

Target	Seed/arm	Blocks	Highest-coverage input
cJSON	0 / T25	789	`[[[[[1,[[8[....8`
cJSON	3 / ALPHA004	831	`[[[[[2,[[[[[1...`
libxml2	3 / LIBXML2-400	4382	`<:>/..<P.&.<:....4<b...&H:..<...`
picohttpparser	0 / PICOHTTPPARSER-400	68	`AAk .,T.%...?...4.b.  .lr......`

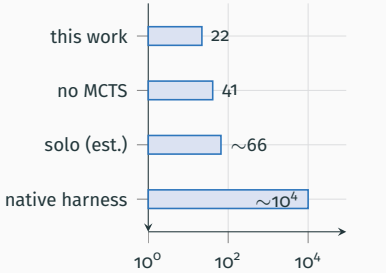
- all four cJSON seeds, three different arms, converge on nested arrays
- libxml2: nested tags; coverage 1416 → 5281 blocks
- structural byte rate climbs from chance to 0.90 / 0.73 / 0.73 (cJSON / libxml2 / picohttpparser)
- picohttpparser: saturates at 81 blocks

Performance degrades after a while



- every target *ends below its peak* (peaks at 24 / 31 / 11 % of the budget)
- MCTS action visit entropy collapses somewhat

Limitations and threats to validity



the gap to native is ∼3 orders of magnitude

- **throughput is very low**: emulation and model-based RL require high execution resources, slowing down fuzzing
- **no conventional-fuzzer baseline**: no AFL++/libFuzzer run — the results show feasibility, not competitiveness
- **high experiment variance**: only 5 of 13 trained arms separate cleanly from the baseline, more runs are needed
- **single fixed-length seed**: full corpus formulation with multiple seeds not evaluated

Conclusions

RQ1 **Yes** — useful policies on all six targets, up to an OPC UA session and invented JSON/XML nesting.

RQ2 **Both work; corpus-level is more flexible** — classical CGF without hand-crafted operators.

RQ3 **Planning helps early, fades late** — simulation count barely moves final coverage at this scale.

Takeaway

Model-based deep RL can drive a fuzzer end-to-end. The main obstacle is execution throughput.

Future work: full seed corpus (action-space scaling via Sampled/Gumbel MuZero, EfficientZero V2) · hybrid CGF + RL loop · larger targets · better implementation

Questions?

1.77× random search at an identical budget

`[[[[[2,[[[[[1...` — input structure the agent invented

the first fuzzer that plans with a *learned* model of its target

Backup

Backup: campaign inventory

Target	Arm	n	Iter.	T	ξ	Cov.	Range	Hours
cJSON	`ALPHA004`	4	200	sched.	0.04	876	829–880	22.0
cJSON	`ALPHA010`	4	200	sched.	0.1	817	666–897	21.8
cJSON	`BASE`	4	200	sched.	1	645	613–668	21.3
cJSON	`CTRL-RAND`	4	200	sched.	1	542	540–543	11.3
cJSON	`EDGE`	4	200	sched.	1	708	660–730	21.7
cJSON	`MEDIUM`	4	200	sched.	1	680	656–754	20.8
cJSON	`NOCOV`	4	200	sched.	1	755	678–799	21.1
cJSON	`NOSEED`	4	200	sched.	1	669	610–754	21.6
cJSON	`REUSE4`	4	200	sched.	1	734	684–866	30.6
cJSON	`SIMS100`	4	197–200	sched.	1	640	620–671	11.5
cJSON	`SIMS50`	4	200	sched.	1	687	629–697	7.6
cJSON	`SMALL`	4	200	sched.	1	683	635–695	21.8
cJSON	`T15`	4	200	1.5	1	936	926–1120	21.0
cJSON	`T25`	4	192–198	2.5	1	958	938–969	20.8
cJSON	`W0`	4	200	sched.	1	641	636–648	19.5
libxml2	`LIBXML2-400`	4	400	2	0.039	5281	5085–5419	42.1
picohttp.	`PICO...-400`	4	400	2	0.039	81	80–81	40.7

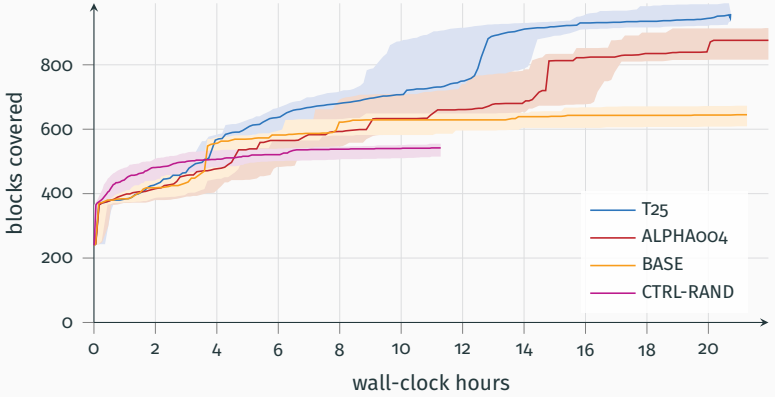
Backup: deep-seed rate (primary cJSON metric)

Percentage of episodes whose best seed reaches the 99th percentile of the CTRL-RAND distribution:

Arm	Mean
CTRL-RAND	1.08
SIMS50	68.68
BASE	76.66
NOCOV	77.98
T25	80.21
W0	81.63
EDGE	82.51
REUSE4	**87.46**

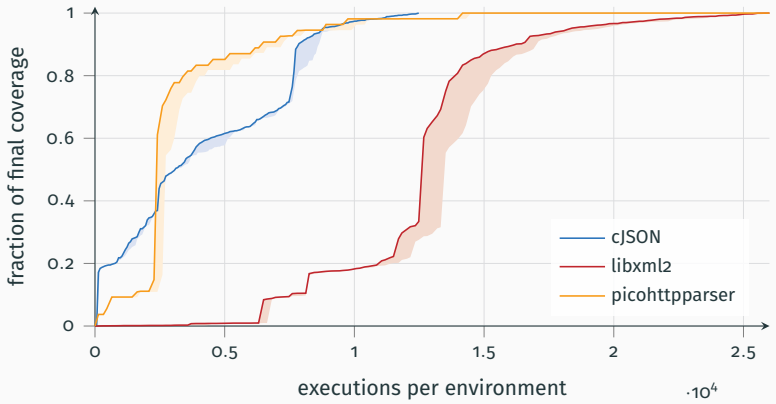
- complements final union coverage: measures how reliably episodes find deep seeds, not just the best one
- every trained arm sits between 68.7 and 87.5 %; random at 1.08 %
- note the ranking differs from final coverage (REUSE4 leads here, T25 there)

Backup: coverage against wall-clock (cJSON)



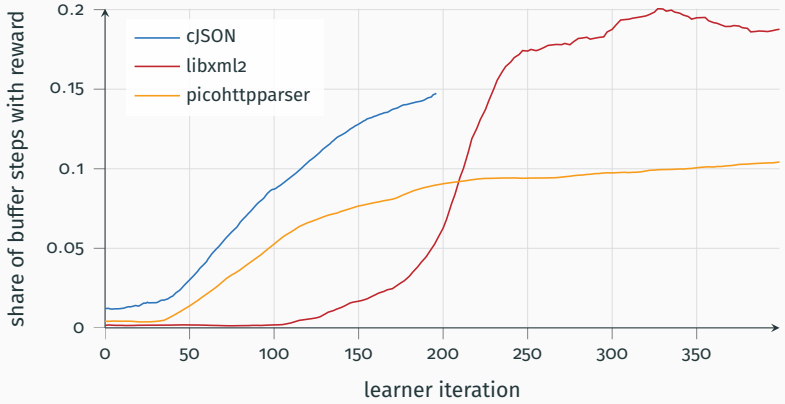
The random control finishes its budget in half the time — search costs a factor of ∼2 in throughput.

Backup: saturation



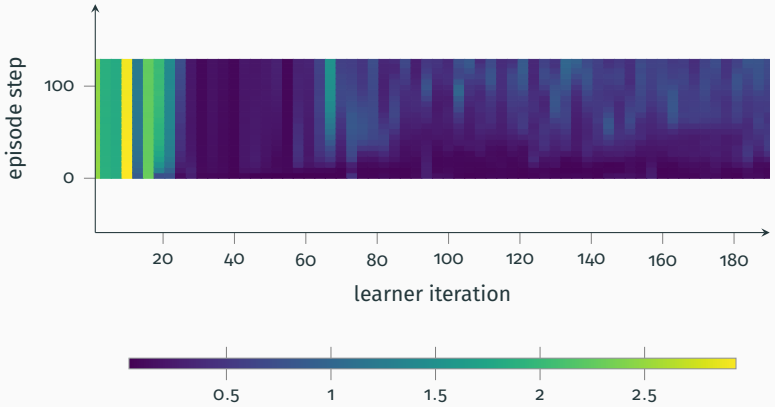
picohttpparser reaches 90 % of its final coverage within the first quarter of the budget and adds nothing in the last quarter.

Backup: reward density in the replay buffer



The share of reward-carrying transitions climbs throughout training (cJSON 0.012 → 0.147) — the agent increasingly picks actions that still find new coverage.

Backup: per-position entropy collapse (cJSON)



MCTS root visit entropy per episode step. Entropy falls after $\sim$23 iterations; early positions collapse first while later steps keep exploring.

Backup: throughput accounting and model sizes

Throughput (derived: 128 envs $\times$ execs/env / hours; all under 8-way GPU sharing on one L40S)

Arm	Total execs	Execs/s
cJSON `BASE` (200 sims)	1.66M	$\sim$22
cJSON `SIMS100`	1.66M	$\sim$40
cJSON `SIMS50`	1.66M	$\sim$61
cJSON `CTRL-RAND` (no search)	1.66M	$\sim$41
libxml2 / picohttp. main	3.33M	$\sim$22

solo (uncontended): roughly 3$\times$ these numbers (96 s vs. 300 s per iteration)

Model parameters (Small / Medium presets)

Target	Small	Medium
cJSON	0.70M	1.88M
libxml2	2.28M	5.03M

observation sizes: 8247 (cJSON: 4096 buckets + 16-byte seed), 32 871 (libxml2)